

REST 練習 5：Query Parameter 篩選

Lecture

rs04 的路徑參數解決的是「指定唯一一筆資源」，但很多時候呼叫端要的是「集合裡符合某些條件的一部分」——例如「只看 AA 航空公司的航班」「只看 2019 年以後的航班」。這種「對集合做篩選」的需求，REST 的慣例是用 **Query Parameter** (URL ? 後面的 **key=value**)，不是路徑參數。

路徑參數 vs Query Parameter 的本質差異：路徑參數回答「你要哪一個資源 / 哪一種資源」，是 URI 結構的一部分，通常對應到資料的**識別碼**；Query Parameter 回答「在這個集合裡，你想要哪些 / 怎麼排序 / 看第幾頁」，是附加在集合請求上的**條件**，同一個 URI 路徑可以有無限多種 Query Parameter 組合，都還是同一個「資源」（這個集合本身）。

篩選條件通常是可選的（呼叫端可以不帶任何條件、帶一個、或帶好幾個組合），這題示範的「布林旗標 OR」寫法就是為了優雅處理「這個條件到底有沒有被指定」，避免程式裡出現一長串 **IF...ELSEIF** 排列組合。

這題也是本課程第一次認真討論「篩選出 0 筆」該怎麼回應——這是很多人設計 REST API 時容易忽略的細節：**GET** 一個集合，不管篩選條件多嚴苛，只要 **URI** 本身有效，就應該回 **200 + 空陣列**，而不是 **404**（404 要留給「呼叫端指定的識別碼本身就是錯的」這種情況）。真實世界的 REST API 通常還會有分頁（**?page= / ?limit=**）、排序（**?sort=**）等更多 Query Parameter 慣例，這門課因為篇幅考量沒有涵蓋，有興趣可以自己延伸。

學習目標

- 理解 URI 路徑參數 (rs04 的 **{carrid}/{connid}/{fldate}**) 跟 **Query Parameter** (**?carrid=AA&connid=17**) 的本質差異：路徑參數定位「哪一個資源」，Query Parameter 是對「一個集合」做篩選/排序/分頁等附加條件，兩者都取得到值，但語意完全不同
- 用 **IF_REST_REQUEST~HAS_URI_QUERY_PARAMETER()** 判斷某個篩選條件**有沒有被呼叫端提供**，用 **GET_URI_QUERY_PARAMETER()** (單數) 取單一 Query Parameter 的值
- 學會「多個篩選條件都是可選、可任意組合」時，不用寫一長串 **IF ... ELSEIF ...** 或動態串接 **WHERE** 字串，改用 **WHERE (欄位 = @變數 OR @有給旗標 = @abap_false)** 這種**條件式短路寫法**：每個篩選欄位都配一個「有沒有提供」的布林變數，沒提供時旗標是 **abap_false**，讓 **OR** 那半邊恆真，等於忽略這個篩選條件
- 分辨兩種不同的「查無資料」情境該對應什麼 HTTP 狀態碼：
 - 篩選條件本身指向一個不存在的資源 (例如 **carrid=ZZ**，**ZZ** 根本不是任何航空公司代碼) → 這是**呼叫端的輸入錯誤**，回 **404**
 - 篩選條件都合法，只是剛好篩出 0 筆資料 (例如 **fromdate=20991231**，**AA** 存在但沒有那麼晚的航班) → 這是**正常的空結果**，回 **200 + 空陣列 []**，不是錯誤

為什麼這兩種「查無資料」要分開處理

rs04 的單筆查詢用完整主鍵定位「一筆」資源，查不到就是那筆資源不存在，回 404 很自然。但 rs05 的 **/flights** 是**集合查詢**，篩選條件是「附加在集合上的條件」，不是「資源的身分」——集合本身永遠存在（**/flights** 這個端點是有效的），只是篩選結果可能剛好是空的。如果篩選合法但空結果也回 404，呼叫端會很困惑：到底是「這個 API 路徑錯了」還是「這個查詢條件太窄」？

這題刻意示範折衷做法：把 `carrid` 當作「必須先驗證存在性的識別碼」（因為它像是在講「這個航空公司」），提供了就先查 `SCARR` 確認真的有這家航空公司，不存在就是呼叫端傳錯資料、404；但 `connid` / `fromdate` 這種「篩選型」條件，即使篩出 0 筆也視為正常查詢結果，回 200。這不是絕對規則，而是「識別碼型」跟「篩選型」參數在 REST 設計上常見的處理差異，實務上可以依團隊慣例調整。

為什麼篩選條件選日期、不選票價

草稿版本原本想用 `minprice`（票價下限）當第三個篩選條件，示範數值型的「範圍比較」（`price >= @lv_minprice`），但實測這套訓練資料的 `SFLIGHT` 票價是綁在 `CARRID` + `CONNID` 這個航班代號上，同一個航空公司底下所有航班的票價幾乎都相同（例如 `AA` 全部 26 筆不管哪個 `CONNID`、哪個日期，票價都是 422.94）。這代表 `carrid=AA&minprice=500` 這種組合會直接篩成 0 筆——不是程式錯，而是這組訓練資料本身在「同一家航空公司內」沒有價格差異可篩，示範不出「多條件組合篩出有意義子集」的效果。改用 `fromdate`（`fldate >= 門檻日期`）解決了這個問題：同一個 `CARRID/CONNID` 底下有十幾筆不同日期的航班紀錄，日期篩選天然就有鑑別度。

為什麼「多條件組合」同時保留 `connid` 跟 `fromdate` 兩種示範

`carrid/connid` 是等值篩選（`=`），`fromdate` 是範圍篩選（`>=`），這是兩種本質不同的篩選語意，各自的「多條件組合」示範長相也不一樣，值得分開各展示一次：

- `carrid=AA&connid=17`（等值 + 等值）：鎖定的是「這一條特定航線」，結果一定全部同一個 `carrid/connid`，只有 `fldate` 不同——一看就懂「這是同一條航線的所有班期」，適合當入門範例
- `carrid=AA&fromdate=20190101`（等值 + 範圍）：篩出「這家航空公司從某天開始的所有航班」，結果會橫跨多個不同 `connid`、多個不同 `fldate`——這是正確、預期的行為，不是 bug，範圍篩選本來就會篩出「一個區間」而不是「一個值」（2026-07-12 第一次示範這個範例時曾被誤會成篩選壞掉，其實是對「範圍篩選會回傳一堆不同值」這件事還不熟悉）

`fromdate` 除了示範範圍篩選，也重用了 `rs04` 教過的 `CONV dats( ... )` 轉型手法

（`GET_URI_QUERY_PARAMETER` 拿回來是不帶短橫線的 8 碼數字字串，要轉成 `DATS` 才能跟 `FLDATE` 比較大小），跟 `carrid/connid` 直接拿 `STRING` 比較是不同的處理路徑，值得對照著看。

事前準備

ADT 端物件已由課程準備好（`$TMP`）：

- `ZCL_RS05_APP`——Application Class，router 掛一個資源 `/flights`
- `ZCL_RS05_FLIGHTS`——集合查詢 + 三個可選 Query Parameter 篩選：`carrid`（航空公司代碼，若提供但不存在回 404）、`connid`（航班代號）、`fromdate`（起始日期下限，`fldate >= fromdate`）

題目需求（對照已建好的答案物件）

1. `ZCL_RS05_APP` 的 `GET_ROOT_HANDLER`：跟 `rs02` 起的寫法一樣，router 掛一條路徑

```
``abap DATA(lo_router) = NEW cl_rest_router( ). lo_router->attach( iv_template =
'/flights' iv_handler_class = 'ZCL_RS05_FLIGHTS' ). ro_root_handler = lo_router. ``
```

1. `ZCL_RS05_FLIGHTS` 的 `GET`：

```
``abap DATA(lv_has_carrid) = mo_request->has_uri_query_parameter( 'carrid' ). DATA(lv_has_connid) =  
mo_request->has_uri_query_parameter( 'connid' ). DATA(lv_has_fromdate) = mo_request->  
>has_uri_query_parameter( 'fromdate' ).
```

```
DATA(lv_carrid) = COND string( WHEN lv_has_carrid = abap_true THEN mo_request->  
>get_uri_query_parameter( 'carrid' ) ELSE ` ` ). DATA(lv_connid) = COND string( WHEN lv_has_connid  
= abap_true THEN mo_request->get_uri_query_parameter( 'connid' ) ELSE ` ` ). DATA(lv_fromdate)  
= COND dats( WHEN lv_has_fromdate = abap_true THEN CONV dats( mo_request->get_uri_query_parameter(  
'fromdate' ) ) ELSE '00000000' ).
```

```
IF lv_has_carrid = abap_true. SELECT SINGLE carrid FROM scarr WHERE carrid = @lv_carrid INTO  
@DATA(lv_carrid_check). IF sy-subrc <> 0. RAISE EXCEPTION TYPE cx_rest_resource_exception EXPORTING  
status_code = cl_rest_status_code=>gc_client_error_not_found request_method =  
if_rest_message=>gc_method_get textid = cx_rest_resource_exception=>resource_not_found. ENDIF. ENDIF.
```

```
SELECT carrid, connid, fldate, price, currency FROM sflight WHERE ( carrid = @lv_carrid OR @lv_has_carrid =  
@abap_false ) AND ( connid = @lv_connid OR @lv_has_connid = @abap_false ) AND ( fldate >=  
@lv_fromdate OR @lv_has_fromdate = @abap_false ) ORDER BY carrid, connid, fldate INTO TABLE  
@DATA(lt_flight) UP TO 20 ROWS.
```

```
DATA(lv_json) = /ui2/cl_json=>serialize( data = lt_flight pretty_name = /ui2/cl_json=>pretty_mode-camel_case  
 ).
```

```
DATA(lo_entity) = mo_response->create_entity( ). lo_entity->set_string_data( lv_json ). lo_entity->  
>set_content_type( if_rest_media_type=>gc_appl_json ). ``
```

fromdate 用 `CONV dats( ... )` 轉型——跟 rs04 單筆查詢的 `fldate` 路徑參數一樣，

`GET_URI_QUERY_PARAMETER( )` 拿回來的是 `STRING` (不帶短橫線的 8 碼數字，例如 `'20190101'`)，要轉成 `DATS` 型別才能拿去跟 `SFLIGHT-FLDATE` 比較大小；如果輸入不是合法的 8 碼日期，這題目前的程式碼不會擋下來 (跟 rs04 思考題第 1 點是同一類已知限制，rs07 才第一次實測發現 `CONV dats( ... )` 對非數字字串根本不會拋出例外，統一防護留到 rs07 才補上，見該題 [read_keys](#) 的說明)。

這裡刻意示範的技巧是「布林旗標 **OR**」，不是直接對變數用 **IS INITIAL**：一開始的草稿曾想寫 `WHERE ( carrid = @lv_carrid OR @lv_carrid IS INITIAL )`，省掉 `lv_has_carrid` 這三個旗標變數，但這套 sap-adt MCP 所連的系統版本對 `STRING` 型別的 `host` 變數直接在 `WHERE` 子句用 **IS INITIAL** 會報語法錯誤 (`Only elementary types for host variables are permitted in expressions`)，只有先用 `HAS_URI_QUERY_PARAMETER( )` 存成獨立的布林變數、`WHERE` 子句改比較這個布林旗標，才穩定可以啟用——這是這套系統版本的限制，不是 Open SQL 語法本身不支援 **IS INITIAL**。

UP TO 20 ROWS 這個上限跟前面的 **WHERE** 篩選條件是各自獨立的機制，順序上是「先篩選、後截斷」：`WHERE` 子句先決定「符合條件的有哪些」，**UP TO 20 ROWS** 才對這個已篩選、已排序的結果集取前 20 筆——如果篩選後符合的筆數本來就比 20 少 (例如 `connid=17` 只有 13 筆)，**UP TO 20 ROWS** 不會有任何影響；但如果篩選後符合的筆數超過 20 (例如只篩 `carrid=AA` 有 26 筆符合)，最終回傳的還是只有前 20 筆，篩選條件本身「有篩到 26 筆」跟 API 最終「回傳幾筆」是兩件事，別搞混了。

SICF 掛載步驟 (比照 rs01~rs04)

沿用既有的 `/sap/bc/zrest_training` 分類節點，這題只要在其底下再加一個子節點：

1. SICF 交易碼，在 `zrest_training` node 上右鍵 → **New Sub-Element**，Service Name 填 `rs05`
2. Handler List 掛 `ZCL_RS05_APP`
3. Activate Service
4. 用 `SPROX_HTTP_REQUEST` (或連得到內網時用瀏覽器) 測幾種組合：**URL 要填完整網址 (含 `http://` 與主機:port)**，不能只填 `/sap/bc/...` 相對路徑——`SPROX_HTTP_REQUEST` 雖然是在 Application Server 上跑的 ABAP 程式、位於內網，但它執行的是一支真正的 HTTP request (呼叫 ICM 的 HTTP Port)，不是程式內部呼叫，沒給 host:port 會找不到這個 API endpoint
 - 不帶任何 Query Parameter：`http://<主機>:<port>/sap/bc/zrest_training/rs05/flights?sap-client=130` (跟 rs03/rs04 的集合查詢結果一樣，UP TO 20 ROWS 截斷後 20 筆)
 - 單一篩選：`http://<主機>:<port>/sap/bc/zrest_training/rs05/flights?carrid=AA&sap-client=130` (AA 資料庫實際符合 26 筆，但受 UP TO 20 ROWS 上限截斷，只會回傳前 20 筆)
 - 多條件組合 (等值 + 等值)：`http://<主機>:<port>/sap/bc/zrest_training/rs05/flights?carrid=AA&connid=17&sap-client=130` (鎖定同一條航線，剛好 13 筆，不受 20 筆上限影響，全部回傳)
 - 多條件組合 (等值 + 範圍)：`http://<主機>:<port>/sap/bc/zrest_training/rs05/flights?carrid=AA&fromdate=20190101&sap-client=130` (同一家航空公司從 2019-01-01 起的所有航班，剛好 20 筆，橫跨不同 connid、不同 fldate，這是範圍篩選正確的行為，不是截斷造成的巧合——資料庫真實比對到的就是剛好 20 筆)
 - 刻意測一個不存在的航空公司代碼，確認回應是 404：`http://<主機>:<port>/sap/bc/zrest_training/rs05/flights?carrid=ZZ&sap-client=130`
 - 刻意測一個合法但篩不出資料的組合，確認回應是 200 + []：`http://<主機>:<port>/sap/bc/zrest_training/rs05/flights?carrid=AA&fromdate=20991231&sap-client=130`

預期輸出 (範例，皆已對 SFLIGHT 實際查詢驗證過)

?carrid=AA (資料庫實際符合 26 筆，受 UP TO 20 ROWS 截斷只回傳前 20 筆)：

```
[{"carrid":"AA","connid":17,"fldate":"2018-09-27","price":422.94,"currency":"USD"},
...]
```

?carrid=AA&connid=17 (等值 + 等值，鎖定單一航線，剛好 13 筆，不受截斷影響，全部回傳)：

```
[{"carrid":"AA","connid":17,"fldate":"2018-09-27","price":422.94,"currency":"USD"},
{"carrid":"AA","connid":17,"fldate":"2018-10-29","price":422.94,"currency":"USD"},
...(共 13 筆，全部 connid 都是 17，只有 fldate 不同)]
```

?carrid=AA&fromdate=20190101 (等值 + 範圍，篩出 2019-01-01 起的所有 AA 航班，剛好 20 筆，橫跨 connid=17、connid=64 等多條航線)：

```
[{"carrid":"AA","connid":17,"fldate":"2019-01-01","price":422.94,"currency":"USD"},
{"carrid":"AA","connid":17,"fldate":"2019-02-02","price":422.94,"currency":"USD"},
...(共 20 筆，connid 跟 fldate 都不同，這是範圍篩選正確的結果，不是等值篩選)]
```

?carrid=ZZ (ZZ 不是任何航空公司代碼) : HTTP 狀態碼 404 。

?carrid=AA&fromdate=20991231 (合法但篩不出資料) : HTTP 狀態碼 200 , 回應內容是空陣列 [] 。

團隊實務備註

- WHERE (欄位 = @變數 OR @有給旗標 = @abap_false) 這個寫法要搭配獨立的「有沒有提供」布林旗標，不能只看變數本身是不是初始值——例如這題 fromdate 不提供時預設是 '00000000'，如果只用「變數是否為初始值」判斷剛好也能正確運作（因為沒有航班日期會比 '00000000' 還早），但這只是巧合；如果哪天篩選欄位的「合法值」剛好包含初始值本身有意義的情境（例如某個 flag 欄位篩 = '' 本身就有意義），就必須靠獨立旗標才能正確分辨，不能偷懶省略
- GET_URI_QUERY_PARAMETERS (複數，這題沒用到) 可以一次拿全部 Query Parameter 存成 TIHTTPNPV 表格，適合「篩選欄位數量不固定、要用迴圈動態組裝」的情境；這題篩選欄位固定只有三個，用單數版本個別取值比較直覺，跟 rs04 選 GET_URI_ATTRIBUTE 單數版本是同樣的考量
- 傳統 ABAP 常見的「動態組 WHERE 字串」(CONCATENATE lv_where '... AND field = ...' INTO lv_where，再用 SELECT ... WHERE (lv_where) 動態語法) 有 SQL Injection 風險、可讀性也差，這題示範的「布林旗標 OR」寫法同樣達到「條件可選」的效果但完全型別安全，不用組字串
- 設計篩選條件時要先看真實資料分布，不能只看欄位型別決定要不要拿來篩：這題原本想用 PRICE (數值型) 示範範圍篩選，直覺上「數值欄位很適合當範圍篩選」，但沒注意到這套訓練資料的票價是綁在航班代號上，同一家航空公司內幾乎沒有價格差異，篩選欄位選錯導致範例「合法但篩不出東西」的機率大增，改選 FLDATE (同一航班有十幾個不同日期) 才是真正有鑑別度的篩選欄位
- 挑選「多條件組合」的教學範例時，優先用等值篩選 (如 connid) 而不是範圍篩選 (如 fromdate) : 範圍篩選天生就會回傳「一堆不同值」的結果，這是正確行為，但拿來當第一次示範「篩選是怎麼運作的」教材時容易造成「為什麼結果還是有一堆不一樣的東西」的誤會；等值篩選鎖定單一值，結果一看就懂，適合當入門範例，範圍篩選則適合另外獨立示範 (這題留在「合法但篩不出資料」的空結果測試裡)

思考題

1. 如果呼叫端同時給 carrid=AA 跟 connid=9999 (AA 存在，但 9999 這個航班代號在 AA 底下不存在)，篩選結果會是什麼？這種情況要不要也回 404？為什麼這題選擇讓它回 200 + 空陣列 (提示：對照學習目標第三點的判斷原則，connid 是「篩選型」還是「識別碼型」參數？)
2. 這題的 carrid 驗證是先跑一次 SELECT SINGLE ... FROM scarr，再跑主查詢的 SELECT ... FROM sflight——等於同一個請求打了兩次資料庫。如果 SFLIGHT 資料量很大、這個驗證成為效能瓶頸，有沒有辦法只用一次查詢就同時達到「驗證存在性」跟「取得篩選結果」？
3. HAS_URI_QUERY_PARAMETER('carrid') 判斷「有沒有提供」，GET_URI_QUERY_PARAMETER('carrid') 取值——如果呼叫端傳的是 ?carrid= (key 有給，value 是空字串)，這兩個方法各自會回傳什麼？這種輸入在這題的邏輯下會被當成「有篩選」還是「沒篩選」？
4. 這題的 UP TO 20 ROWS 上限是寫死在程式裡的，跟呼叫端給的篩選條件完全無關——如果呼叫端明明篩選出 26 筆符合條件的資料，API 卻只回傳前 20 筆，呼叫端要怎麼知道「這是被截斷的結果」還是「資料庫真的只有 20 筆」？ (提示：這是真實 REST API 分頁設計要解決的問題，這題故意沒做分頁，只是先讓你注意到這個現象)

答案

見 zcl_rs05_app.clas.abap、zcl_rs05_flights.clas.abap (SAP 端物件 ZCL_RS05_APP / ZCL_RS05_FLIGHTS)。SICF Service 路徑 /sap/bc/zrest_training/rs05。